What is Gradual Typing?

Yudai Tanabe

Tokyo Institute of Technology

Today's Topic

- Background Knowledge
  - Static/Dynamic type checking
- Basic topics of Gradual typing
  - Basics (STLC + any type)
  - Static typing in Gradual Typing
  - Runtime casting in Gradual Typing
- Advanced topic of Gradual typing
  - The type system of GTLC
  - Dynamic semantics of GTLC
  - The state of Gradual Typing

What is Type?

- Types:

What is Type?

- Types:
  - A set of values with a common set of operations, *or*
  - A static property of the program

2 ways of type-checking

- Dynamic type checking
  - Type checking is performed ***during program execution***.
  - Make sure that the operand's type is suitable for the operation
  - e.g.) Python, JavaScript, Ruby, PHP, Perl, …
    Scheme (the target language of EOPL seminar!)

- Static type checking
  - Some or even all type errors are caught ***before program execution***.
  - The type checker is usually part of the compiler, so automatically run during compilation.
  - e.g.) ML-family, Haskell, Java, C, C++, Rust, …

Dynamic type checking

Ex.) Concatenation of a string value and an integer value

- Javascript “String.concat”：Accepted on ECMA-262[1]

```
function myconcat(str) {
    return str.concat("a");
}
var x = 1;
console.log("The program up to this point is executed.");
myconcat(x);
/*
 * $ node concat.js
 * The program up to this point is executed.
 *     return str.concat("a");
 *                ^
 * TypeError: str.concat is not a function
 *     at concat (concat.js:2:16)
 */
```


[1] https://www.ecma-international.org/ecma-262/10.0/index.html#Title

Static type checking

Ex.) Concatenation of a string value and an integer value

- Java “String.concat”：Rejected on open JDK 8

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String correct = "a".concat("a"); // well-typed
        String invalid = "a".concat(1); // ill-typed
        System.out.println(correct);
    }}
/*
$ javac concat.java
Main.java:9: error: incompatible types: Integer cannot be
converted to String
        String invalid = "a".concat(1); // ill-typed
                                    ^
1 error
*/
```

Pros and Cons of *Dynamic* Type Checking

- Pros:
  - Concisely write a program
    - A type has nothing to do with the mainstream of processing
    - No need to solve complex type puzzles
  - Useful for writing programs depend on runtime information
    - e.g) Duck typing
- Cons:
  - Error detection is at runtime
  - Few hints for reading comprehension
  - Slow due to runtime type check

Pros and Cons of *Static* Type Checking

- Pros:
  - Provides useful compile-time information
    - x + y -> (x86) ADD S(0),S(1) *or* FADD S(0),S(1) *?*
  - Reject pointless programs before the execution
    - 1 ++ “a” -> undefined…
  - Contributes to maintainability as the simplest specification
- Cons:
  - Low flexibility - type declaration as program constraint
    - We don’t know future spec. changes is suitable to type spec. => Compatibility problems
  - Type puzzle

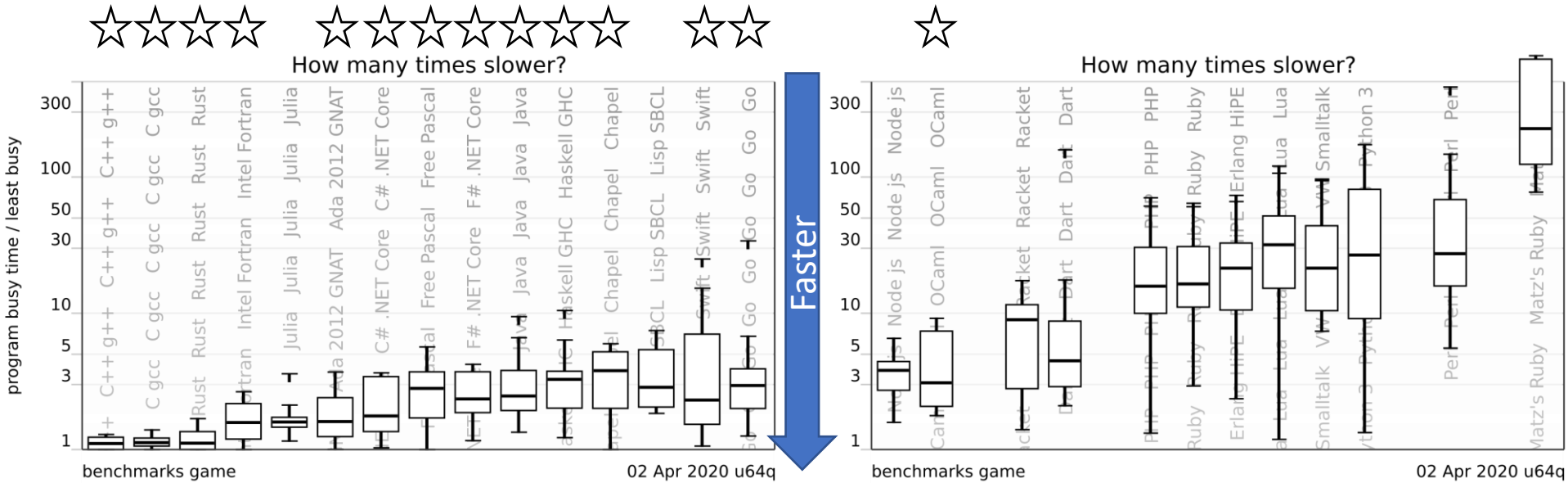
Note: Which Language is fastest?

- The Computer Language Benchmarks Game[1]

10 benchmarks
N-body, Mandelbrot, ..

"How many times slower, the fastest benchmark programs for selected programming language implementations are, compared to the fastest written in any of the programming languages."

☆ Statically-typed language



[1] https://benchmarksgame-team.pages.debian.net/benchmarksgame/which-programs-are-fastest.html

A Motivation of Gradual Typing

- Neither static/dynamic type checking is universally better than the other
- Provide the programmer a choice, without forcing them to switch programming languages
  - Switching to a different Language…
    - Changes the ecosystem and toolchain
    - Demands significantly higher learning costs

Gradual Typing

What is Gradual Typing?

"Smoothly combining static and dynamic type checking <u>in a single language</u> by introducing any*"*

What is Gradual Typing?

*"Smoothly **combining static and dynamic type checking in a single language** by introducing **any**"*

- Ex.) TypeScript: Superset of JavaScript
  - Prog. with type annotation -> Statically type checked
  - Prog. without type annotation -> Dynamcally type checked
    (⇔ with **any** type)
- We can import the following JavaScript code into TypeScript

```
function add(x, y){ return x + y; }
add(1, 2);
```

What is Gradual Typing?

"Smoothly combining static and dynamic type checking in a single language ***by introducing*** **any***"*

- **any** type (dynamic type, ? type, * type)
  - Represents the part of programs to be dynamically type checked
  - Bahaves like a *wildcard* when static type checking

```
function add(x, y){ return x + y; }
// x & y are treated as any.
add(1, 2);
// actual args 1 & 2: number, expected args: any
// any is treated as wildcard, so the programs are accepted.
```

What is Gradual Typing?

*"**Smoothly** combining static and dynamic type checking in a single language by introducing **any**"*

- Gradual typing **evolves codes, *gradually***

```
function add(x, y){ return x + y; }
function add(x:number, y){ return x + y; }
function add(x:number, y:number){ return x + y; }
```

  - We can benefit from static typing as necessary
  - Allows us to add types to existing JavaScript code
  - Integrates typed code with existing JavaScript libraries

Introduction to Gradual Typing

- Gradual Typing:
  - Static typing that treats **any** type as a *wildcard*
  - Dynamic type checking by runtime casting

Note: Assume the small TypeScript-like language

```
(Type) T := num, str, any, T -> T
(Syntax) var, func(function), application, return
          are equivalent to TypeScript
```

Static Typing in Gradual Typing (1)

- Without **any:** Same way as a statically typed language

```
func add1(x: num){ return x + 1; }
            // num -> num
add1(1);    // num
add1("a");  // Ill typed! – num != str
```

- With **any: any** is treated as a **wildcard**!

```
func add1(x: any){ return x + 1; }
            // any -> num
add1(1);    // num
add1("a");  // num
// Because… any can be treated as str
```

```
add1(add1); // num
// Because… any can be treated as any -> num
```

Static Typing in Gradual Typing (2)

- We can also use **any** as part of a complex type
  - Similarly treated like a wildcard

```
func add2(…){…} // (any -> num) -> num
func f1(…){…}   // str -> num

add2(1);  // ill-typed!
// Because… any -> num cannot be treated as num

add2(f1); // num
// Because… any -> num can be treated as str -> num
```

Static Typing in Gradual Typing (3)

- Static typing with **any** is insufficient

```
func id(x: num){ return x; }
            // num -> num

var y: any = “a”; // any

id(y); // num
// Because… num can be treated as any
```

- It’s accepted for static type checking, but statically-unsafe!
  - Violate id’s arguments type specification

Static & Dynamic type cheking, Compensate for Each Other

- Reprint:

> Gradual Typing:
> - Static typing that treats **any** type as a *wildcard*
> - Dynamic type checking by runtime casting

- **Runtime-casting insertion** compensates for static typing

Runtime casting in Gradual Typing (1)

```
t ::= … | <T1➔T2>t // cast of t from T1 to T2
```

- Casts that will succeed

```
<num➔num>1
<num➔any>1
<any➔num>(<num➔any>1)
```

- Casts that will fail

```
<num➔str>1
<num➔num->num>x
<any➔str>(<num➔any>1)
```

Runtime Casting in Gradual Typing (2)

- Insert casts where types are not equal when static typing

```
func id(x: num){ return x; }
var y: any = “a”;
id(y);
```

Static typing

```
func id(x: num){ return x; } // num -> num
var y: any = “a”; // any
id(y); // num
```

Cast insertion

```
func id(x: num){ return x; } // num -> num
var y: any = <str➔any>“a”; // any
id(<any➔num>y); // num
```

Next slide…

Runtime Casting in Gradual Typing (3)

- The runtime cast catches the errors missed by static typing

```
id(<any➔num>y)
--> id(<any➔num>(<str➔any>”a”))
--> Runtime TypeError! // Cast Exception!
```

```
Reprint:
func id(…){…} // num -> num
var y: any = <str➔any>“a”; // any
id(<any➔num>y); // num
```

A Formalization of Gradual Typing

- The Gradually Typed Lambda Calsulus (GTLC)[Siek06]
  - Extends STLC with unknown type $\star$ (= any) and un-annotated functions

Blame labels ℓ	
Base types	$B ::= int \mid bool$
Types	$T ::= B \mid T \to T \mid \star$
Constants	$k ::= true \mid 0 \mid \ldots$
Expressions	$e ::= k \mid x \mid \lambda x{:}T.\, e \mid (e\ e)^{\ell} \mid \lambda x{:}\star.\, e$

  - Blame labels ℓ represent source position information (unique)
    - ℓ are used for dynamic semantics when an error occurs.
    - ℓ have nothing to do with typing

[Siek06] http://scheme2006.cs.uchicago.edu/13-siek.pdf

Type System of GTLC

- The typing rules except for app are the same as STLC.
- 2 key aspects can be seen in T-app rule

$$\frac{\Gamma \vdash e_1 : T_1 \qquad \Gamma \vdash e_2 : T_2 \qquad fun(T_1) = T_{11} \to T_{12} \qquad T_2 \sim T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} : T_{12}}$$ T-app on GTLC

Typing System of GTLC

- The typing rules except for app are the same as STLC
- 2 key aspects can be seen in T-app rule

$$\frac{\Gamma \vdash e_1 : T_1 \qquad \Gamma \vdash e_2 : T_2 \qquad fun(T_1) = T_{11} \to T_{12} \qquad T_2 \sim T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} : T_{12}}\ \text{T-app on GTLC}$$

$$\frac{\Gamma \vdash e_1 : T_{11} \to T_{12} \qquad \Gamma \vdash e_2 : T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} : T_{12}}\ \text{T-app on STLC}$$

Typing System of GTLC

- The typing rules except for app are the same as STLC
- 2 key aspects can be seen in T-app rule

$$\frac{\Gamma \vdash e_1 : T_1 \qquad \Gamma \vdash e_2 : T_2 \qquad fun(T_1) = T_{11} \to T_{12} \qquad T_2 \sim T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} : T_{12}}$$

T-app on GTLC

Function matching fun

Type consistency "$\sim$"

$$\frac{\Gamma \vdash e_1 : T_{11} \to T_{12} \qquad \Gamma \vdash e_2 : T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} : T_{12}}$$

T-app on STLC

Function Matching fun

- fun captures a situation where the type of operator is unknown

Function Matching

$$fun(T_{11} \to T_{12}) = T_{11} \to T_{12}, \qquad fun(\star) = \star \to \star$$

$$\frac{\Gamma \vdash e_1 : T_1 \qquad fun(T_1) = T_{11} \to T_{12} \qquad \Gamma \vdash e_2 : T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} : T_{12}}$$

Case. 1

$$\frac{\Gamma \vdash e_1 : T_{11} \to T_{12} \qquad \Gamma \vdash e_2 : T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} : T_{12}}$$

Case. 2

$$\frac{\Gamma \vdash e_1 : \star \qquad \Gamma \vdash e_2 : T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} : \star}$$

Type Consistency "∼"

- "∼" captures the behavior of ⋆ (= any) as a wild card
  - The argument type can be any other type masked by ⋆

$$\frac{\Gamma \vdash e_1 : T_1 \qquad \Gamma \vdash e_2 : T_2 \qquad fun(T_1) = T_{11} \to T_{12} \qquad T_2 \sim T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} : T_{12}}$$

- Ex.)
  - $num \sim num$
  - $num\ !\sim\ bool$
  - $any \sim num$
  - $num \to any \sim any \to bool$
  - $num \to num\ !\sim any \to bool$

Type Consistency

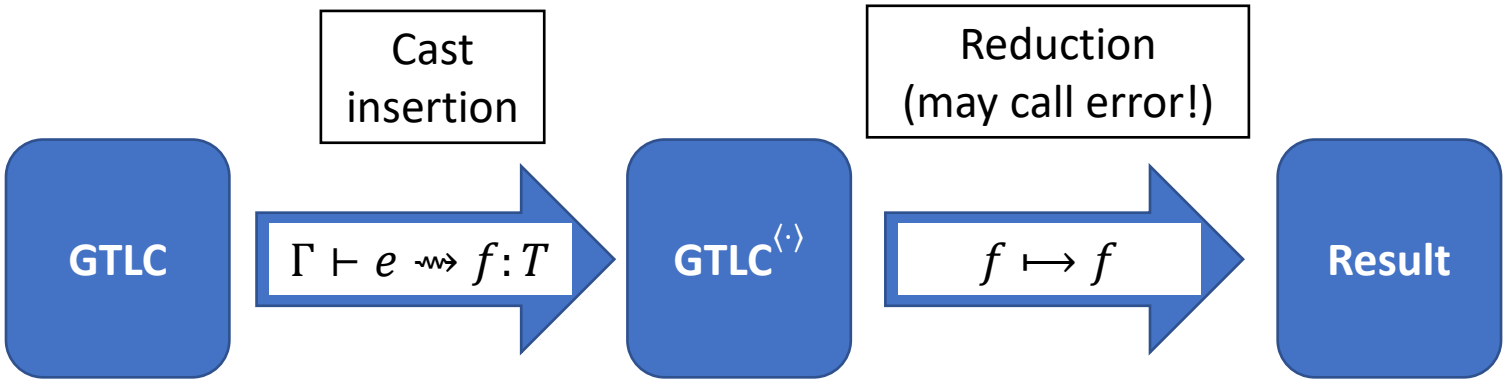
$$\frac{}{T \sim \star} \qquad \frac{}{\star \sim T}$$

$$\frac{}{B \sim B}$$

$$\frac{T_1 \sim T_3 \qquad T_2 \sim T_4}{T_1 \to T_2 \sim T_3 \to T_4}$$

Dynamic Semantics Model of GTLC

- GTLC's dynamic semantics has two phases:
  - Cast insertion
  - Reduction



$\Gamma \vdash e : T$ $\qquad$ $\Gamma \vdash_C f : T$

Expressions	$f ::= \cdots \mid \langle T \Rightarrow^{\ell} T \rangle f \mid \mathrm{blame}_T \ell$
Results	$r ::= \cdots \mid \mathrm{blame}_T \ell$

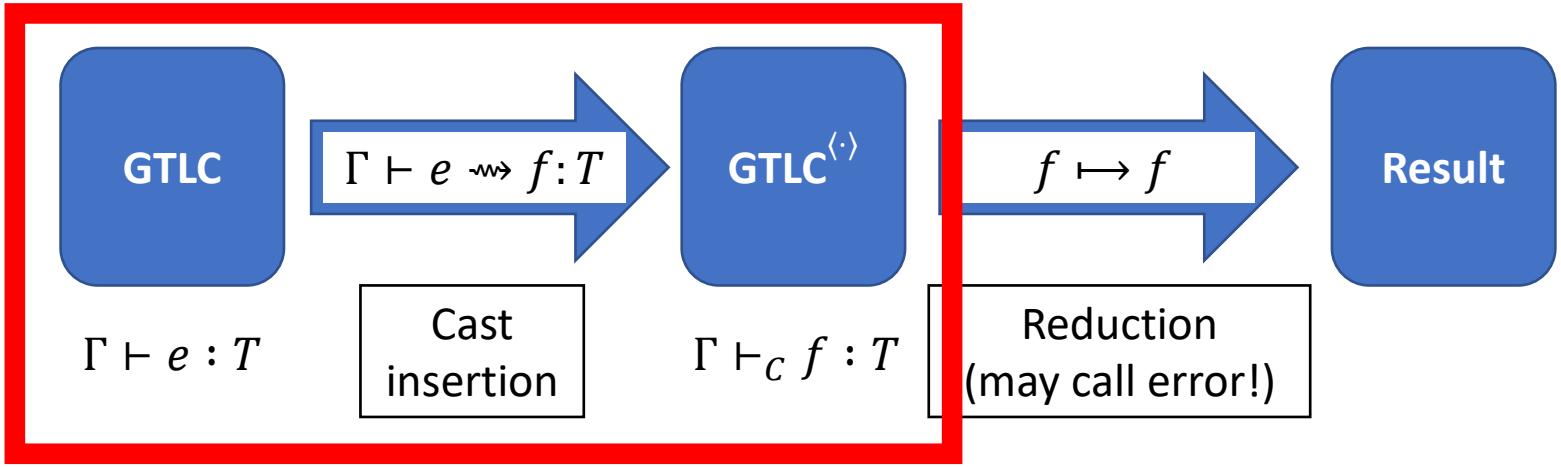
Dynamic Semantics Model of GTLC

- Ex.) Cast insertion of application

$$\frac{\Gamma \vdash e_1 \rightsquigarrow f_1 : T_1 \qquad \Gamma \vdash e_2 \rightsquigarrow f_2 : T_2 \qquad fun(T_1) = T_{11} \to T_{12} \qquad T_2 \sim T_{11}}{\Gamma \vdash (e_1\ e_2)^{\ell} \rightsquigarrow \left(\langle T_1 \Rightarrow^{\ell} T_{11} \to T_{12}\rangle f_1\right)\left(\langle T_2 \Rightarrow^{\ell} T_{11}\rangle f_2\right) : T_{12}}$$

GTLC term

GTLC$^{\langle\cdot\rangle}$ term



Dynamic Semantics Model of GTLC

- Ex.) Reduction of casting

Dynamic Semantics $\boxed{f \longmapsto f}$

$$(\lambda x{:}T.\, f)\; v \longmapsto [x := v]f \qquad \text{(BETA)}$$

$$v : B \Rightarrow^{\ell} B \longmapsto v \qquad \text{(IDBASE)}$$

$$v : \star \Rightarrow^{\ell} \star \longmapsto v \qquad \text{(IDSTAR)}$$

$$v : G \Rightarrow^{\ell_1} \star \Rightarrow^{\ell_2} G \longmapsto v \qquad \text{(SUCCEED)}$$

$$v : G_1 \Rightarrow^{\ell_2} \star \Rightarrow^{\ell_2} G_2 \longmapsto \texttt{blame}_{G_2}\, \ell_2 \quad \text{if } G_1 \neq G_2 \qquad \text{(FAIL)}$$

$$(v_1 : T_1 \to T_2 \Rightarrow^{\ell} T_3 \to T_4)\; v_2 \longmapsto v_1\; (v_2 : T_3 \Rightarrow^{\ell} T_1) : T_2 \Rightarrow^{\ell} T_4 \qquad \text{(APPCAST)}$$

$$v : T \Rightarrow^{\ell} \star \longmapsto v : T \Rightarrow^{\ell} G \Rightarrow^{\ell} \star \quad \text{if } T \neq \star, T \neq G, T \sim G \qquad \text{(GROUND)}$$

$$v : \star \Rightarrow^{\ell} T \longmapsto v : \star \Rightarrow^{\ell} G \Rightarrow^{\ell} T \quad \text{if } T \neq \star, T \neq G, T \sim G \qquad \text{(EXPAND)}$$

$$F[f] \longmapsto F[f'] \quad \text{if } f \longmapsto f' \qquad \text{(CONG)}$$

$$F[\texttt{blame}_{T_1}\, \ell] \longmapsto \texttt{blame}_{T_2}\, \ell \quad \text{if } \vdash F : T_1 \Rightarrow T_2 \qquad \text{(BLAME)}$$

Dynamic Semantics Model of GTLC

- Ex.) Reduction of casting

Dynamic Semantics $\boxed{f \longmapsto f}$

$$v : G \Rightarrow^{\ell_1} \star \Rightarrow^{\ell_2} G \longmapsto v \qquad \text{(SUCCEED)}$$

$$v : G_1 \Rightarrow^{\ell_2} \star \Rightarrow^{\ell_2} G_2 \longmapsto \texttt{blame}_{G_2}\, \ell_2 \quad \text{if } G_1 \neq G_2 \qquad \text{(FAIL)}$$

$$(v_1 : T_1 \to T_2 \Rightarrow^{\ell} T_3 \to T_4)\ v_2 \longmapsto v_1\ (v_2 : T_3 \Rightarrow^{\ell} T_1) : T_2 \Rightarrow^{\ell} T_4 \qquad \text{(APPCAST)}$$

$$F[f] \longmapsto F[f'] \quad \text{if } f \longmapsto f' \qquad \text{(CONG)}$$

$$F[\texttt{blame}_{T_1}\, \ell] \longmapsto \texttt{blame}_{T_2}\, \ell \quad \text{if } \vdash F : T_1 \Rightarrow T_2 \qquad \text{(BLAME)}$$

Dynamic Semantics Model of GTLC

- Ex.) Reduction of casting

Dynamic Semantics

$$\boxed{f \longmapsto f}$$

$$\Gamma \vdash (e_1\ e_2)^{\ell} \rightsquigarrow \left(\langle T_1 \Rightarrow^{\ell} T_{11} \to T_{12}\rangle f_1\right)\left(\langle T_2 \Rightarrow^{\ell} T_{11}\rangle f_2\right) : T_{12}$$

$$(v_1 : T_1 \to T_2 \Rightarrow^{\ell} T_3 \to T_4)\ v_2 \longmapsto v_1\ (v_2 : T_3 \Rightarrow^{\ell} T_1) : T_2 \Rightarrow^{\ell} T_4 \qquad (\textsc{AppCast})$$

Dynamic Semantics Model of GTLC

- Ex.) Reduction of casting

Dynamic Semantics

$f \longmapsto f$

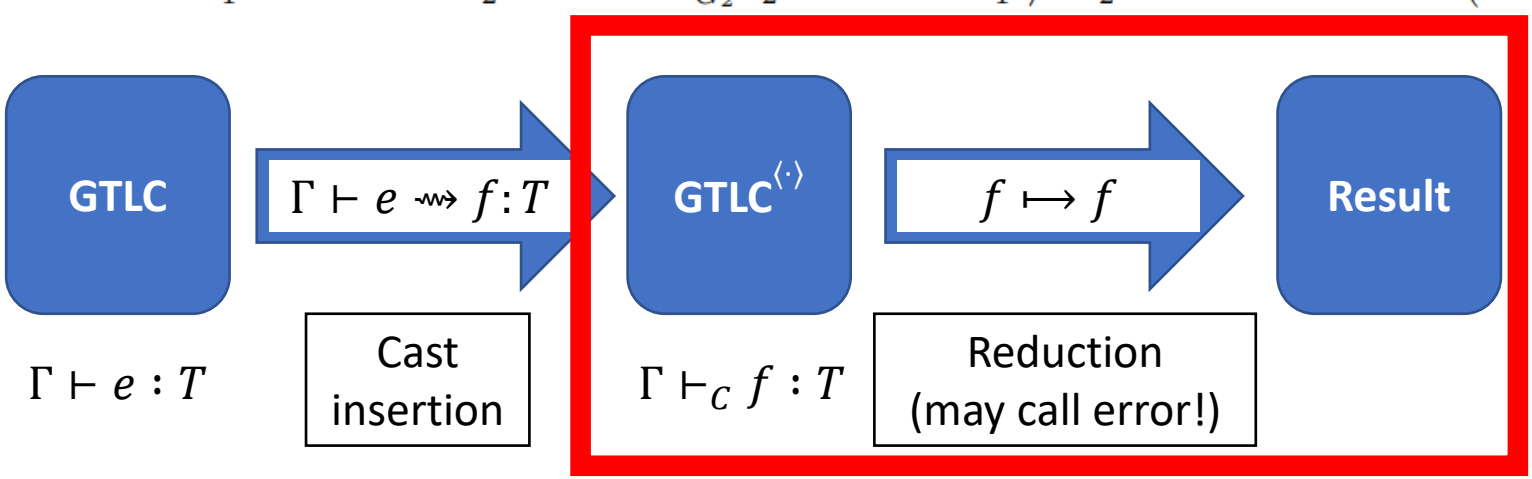
Reduction rules of casting

$$\langle \star \Rightarrow^{\ell_2} T_2 \rangle \langle T_1 \Rightarrow^{\ell_1} \star \rangle v \longmapsto v \qquad \text{if } T_1 = T_2$$

$$\langle \star \Rightarrow^{\ell_2} T_2 \rangle \langle T_1 \Rightarrow^{\ell_1} \star \rangle v \longmapsto \mathrm{blame}_{G_2}\, \ell_2 \qquad \text{if } T_1 \neq T_2$$

$$v : G \Rightarrow^{\ell_1} \star \Rightarrow^{\ell_2} G \longmapsto v \qquad \text{(SUCCEED)}$$

$$v : G_1 \Rightarrow^{\ell_2} \star \Rightarrow^{\ell_2} G_2 \longmapsto \mathtt{blame}_{G_2}\, \ell_2 \qquad \text{if } G_1 \neq G_2 \qquad \text{(FAIL)}$$



The State of Gradual Typing

- Gradual Typing is being considered for many languages
  - JavaScript → TypeScript[Microsoft], Flow[Facebook]
  - Python → mypy / pyre / Cython
  - PHP → Hack[Facebook] / PHP 7.0.0
  - Racket → TypedRacket[Hochstadt08]
  - Dart[Google]
  - Dylan[Apple]
  - Raku[Wall20]
- How much of a gap is there between these implementations and theoretical Gradual Typing?

Q1: Is TypeScript Gradual Typing?

- There's a gap between the TypeScript type system and theoretical Gradual Typing[Siek15]
  - A variable without type annotation has any type
  - Use type consistency when static type checking
  - But… type annotations are *not involved at runtime*.
- The following code pass static type checking in TypeScript
  - The target code doesn't include casts

```
func id(x: num){ return x; }
            // num -> num
var y: any = "a"; // any
id(y); // num
// Because… any can be treated as num
```

TypeScript

Static Type Checking & CodeGen

```
function id(x){ return x; }
var y = "a";
id(y);
```

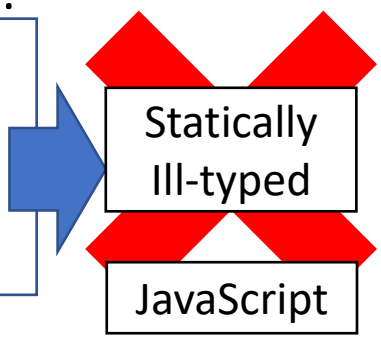
JavaScript

Q1: Is TypeScript Gradual Typing?

- How do we reject the following code?
  - The basic strategy is to make the code fully-annotated
  - **But …** It's hard to do in a 3rd-party library.

```
func id(x: num){ return x; }
                // num -> num
var y: str = "a"; // str
id(y);
```

TypeScript



- A repository for npm interface: DefinitelyTyped[1]
  - **But …** static type checking cannot be done on the inside

[1]https://definitelytyped.org/

```
8   declare var npm: NPM.Static;
9   export = npm;
10
11  declare namespace NPM {
12      export interface Static extends NodeJS.EventEmitter {
13          config: Config;
14          commands: Commands;
15          rollbacks: any[];
16          spinner: Spinner;
```

Q2: Isn't the original language's dynamic type checking a substitute for casting?

- Slower to capture type errors than casting
- Ex.) The left program cannot stop until return x+1
  - With casting, make an error at that point, so no *long side effects*

```
func long(x: num){
    …(so long programs
       with side effects)…
    return x+1;
} // num -> num
var y: any = “a”;
long(y);
```

TypeScript

```
func long(x: num){
    …(so long programs
       with side effects)…
    return x+1;
} // num -> num
var y: any = <str➔any>“a”;
long(<any➔num>y);
```

TypeScript with casting

- Safe TypeScript[Rastogi15]: TS impl. with runtime type checking
  - A 15% runtime overhead for type safety

Script Language Evolution to Gradual Typed Language

1. Define static type on the language
   1. Introduce type annotations into your grammar
   2. Type existing codes and consider what type is needed
   3. Consider the interaction between any and existing language mechanisms
2. Consider dynamic type checking by runtime casting
   - JavaScript → Safe TypeScript[Rastogi15]
   - Python → Reticulated Python[Vitousek14]
   - Racket → TypedRacket[Hochstadt08]

Performance Problems of Gradual typing

- The overhead of sound interoperation between typed and untyped code is quite large
  1. The dynamic checks required to enforce soundness have an inherent runtime cost
  2. The execution of code including the typed and untyped modules is 100 times slower than the execution in Racket[Takikawa16]
- Suggested approach:
  - Reduce the number of runtime casting
  - Optimizing runtime type checking using JIT[1]

Further Readings

Basics:

J. Siek: What is Gradual Typing? (his blog)

Y. Igarashi: Consider the Future of Gradual Typing (jp) (his blog)

S. Hochstadt: Gradual Typing from Theory to Practice (SIGPLAN Blog)

Important Papers:

(Type Consistency) → J. Siek et al.: Gradual Typing for Objects (ECOOP07)

(Metatheory) → J. Siek et al.: Refined Criteria for Gradual Typing (SNAPL15)

(Performance) → A. Takikawa et al.: Is Sound Gradual Typing Dead? (POPL16)

Conclusion

- Gradual Typing consists of …
  - Static typing that treats **any** type as a *wildcard*
  - Dynamic type checking by runtime casting
- I introduced the semantics of Gradual Typing.
  - Static Semantics: Introduce type consistency and any type
  - Dynamic Semantics: Translation into IL including casts and cast reduction
- Discuss the problem of the state of Gradual Typing
  - No run-time type checking in many implementations
  - Large performance overhead